Real-Time Edge Conversational AI System

Amiya Chowdhury

Department of Computer Science and Engineering
National Institute of Technology Rourkela

Real-Time Edge Conversational AI System

Thesis submitted in partial fulfillment

of the requirements for the degree of

Bachelor of Technology

in

Computer Science and Engineering

by

Amiya Chowdhury

(Roll Number: 122CS0067)

based on research carried out

under the supervision of

Prof. Anup Nandy

May, 2026

Department of Computer Science and Engineering

National Institute of Technology Rourkela

Department of Computer Science and Engineering
National Institute of Technology Rourkela

Prof. Anup Nandy
Associate Professor

May 11, 2026

Supervisor's Certificate

This is to certify that the work presented in the thesis entitled *Real-Time Edge Conversational AI System* submitted by *Amiya Chowdhury*, Roll Number 122CS0067, is a record of ongoing original research carried out by him under my supervision and guidance in partial fulfillment of the requirements of the degree of *Bachelor of Technology* in *Computer Science and Engineering* . Neither this thesis nor any part of it has been submitted earlier for any degree or diploma to any institute or university in India or abroad.

Anup Nandy

Dedication

I dedicate this thesis to my beloved family and friends, whose steadfast love and unwavering support have been my guiding force throughout this B.Tech journey. Your encouragement, understanding, kindness and patience have fueled my determination, and your belief in me has been my greatest motivation.

To my family, for being my strength, for the sacrifices made, and for being the giants on whose shoulders I stood and looked into the world, I owe you everything. Thank you for instilling my interest in technology at an early age.

To my friends, for the camaraderie, the laughter, discussions that stretched late nights, and for being there during my highs and lows. Thank you for being the buoys that always guided me to the shore, and let me reach peaks I had never imagined I could reach.

This thesis stands as a testament to the collective efforts and sacrifices of my family and friends. Your love has been the driving force behind every achievement, and I am grateful beyond words for the privilege of having you all in my life.

With heartfelt gratitude,
Amiya Chowdhury

Declaration of Originality

I, *Amiya Chowdhury*, Roll Number *122CS0067* hereby declare that this thesis entitled *Real-Time Edge Conversational AI System* presents my original work carried out as a undergraduate student of NIT Rourkela and, to the best of my knowledge, contains no material previously published or written by another person, nor any material presented by me for the award of any degree or diploma of NIT Rourkela or any other institution. Any contribution made to this research by others, with whom I have worked at NIT Rourkela or elsewhere, is explicitly acknowledged in the thesis. Works of other authors cited in this thesis have been duly acknowledged under the sections “Reference” or “Bibliography”. I have also submitted my original research records to the scrutiny committee for evaluation of my thesis.

May 11, 2026
NIT Rourkela

Amiya Chowdhury

Acknowledgement

I would like to express my sincere gratitude to all those who have supported me in my ongoing efforts on this project. I am particularly indebted to our project supervisor for the final year, ***Professor Anup Nandy***, whose invaluable suggestions and unwavering support have played a pivotal role in guiding me through the process of conducting this research. His encouragement has continually inspired me to work diligently and push boundaries.

In addition, I wish to extend my heartfelt appreciation to the dedicated personnel of the ***Department of Computer Science and Engineering*** for granting me access to the essential equipment and materials necessary for this project.

Lastly, I would like to convey my profound gratitude to my parents and friends for their continuous encouragement throughout my academic journey and during this project. I am also deeply thankful to the **National Institute of Technology, Rourkela** for their backing and for awarding me the opportunity to undertake this project, along with the provision of essential facilities.

May 11, 2026
NIT Rourkela

Amiya Chowdhury
Roll Number: 122CS0067

Abstract

Recent advancements and breakthroughs in Large Language Models and speech processing have enabled the development of intelligent conversational systems. However, most commercially available voice assistants rely on cloud based technologies where the systems are essentially black boxes. On occasions this leads to high latencies, privacy concerns,and dependency on stable connection to the Internet. This project focuses on the design and implementation of a fully on-device conversational voice assistant deployed on resource constrained hardware, with a focus on minimizing latency and optimising resource usage. The proposed system integrates streaming Automatic Speech Recognition, local Language Model inference, and Text to Speech synthesis. A multithreaded pipeline architecture with silence based segmentation and chunked token streaming is implemented to achieve acceptable responsive interaction. Additionally, latency metrics such as ASR to LLM decision latency, Time to First Token, tokens per second, and end to end response latency are measured across test trials for performance evaluation. To achieve this, quantization techniques are utilised on the ASR and language models to enable efficient deployment on the edge hardware. The system functions as as a feasibility study of real time edge conversational AI, potentially contributing towards intelligent sensing and human-robot interaction systems.

Keywords: *Edge AI*; *Voice Assistant*; *Local LLM*; *Conversational Systems*.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Introduction

Voice based assistants are software programs that function to execute actions and other services in response to users' voice input[6, 10]. Popular voice assistants are embedded in devices such as phones and computers. and perform functions such as scheduling reminders, controlling intelligent home devices[10]. However, traditional voice assistants such as conversational agents rely on remote servers for compute intensive speech recognition and language processing, which enables them to provide accurate results, but introduces latency, privacy risks, and dependency on network infrastructure. For embedded and robotics applications, especially in interactive environments, these limitations significantly hinder usability, and sometimes responsiveness and availability. Now that there are efficient open-source speech models and lightweight Large Language Models, it is possible to put conversational intelligence directly on edge devices. Running models on your own computer lowers latency, improves privacy, and makes it possible to work in places where there is no internet or limited bandwidth. This is especially important for intelligent sensing systems, robots, and assistive technologies that need to respond real-time. The goal of this project is to create and test a conversational voice assistant that works completely on the device and has low latency. It will use streaming speech recognition, local LLM inference, and edge text-to-speech synthesis. The system is installed on two different edge devices.

1.2 Motivation

Cloud-based voice assistant solutions use a lot of computing and power resources to give almost perfect answers. However, they need to be connected to the Internet and raise privacy concerns. The main reason for doing this research is to find the feasibility of conversational systems for edge devices that protect users' privacy. Most current solutions use cloud APIs, which don't work well for apps that deal with sensitive data or robots. Moreover, streaming interaction pipelines for ASR and LLM inference are still not well understood in environments with limited hardware resources. In addition to all this,

embedded devices consume less energy compared to full-fledged data centers, and this could serve as a cost-cutting measure for both the end-consumers and the businesses responsible for maintaining these cloud services. This work is also in line with research in intelligent sensing systems and human-robot interaction, where conversational agents need to work well even when there are limits on hardware and latency.

1.3 Objectives

The main goals of this project are: To create a real-time streaming voice assistant pipeline for edge deployment, to add a Speech to Text (STT) method for low-latency streaming speech recognition, to install a quantized LLM locally on edge hardware like the Raspberry Pi or Jetson Nano, to use lightweight methods for on-device TTS synthesis, and finally to measure and evaluate system latency metrics for performance analysis.

1.4 Organization Of Thesis

There are five chapters in this thesis. The five chapters are intended to thoroughly examine the feasibility and create a prototype of a lightweight conversational system on edge device using a Language Model. From identifying the problem to developing and validating a solution, each chapter builds on the one before it to form a cohesive narrative that leads the reader through the research process. The chapters are organized as follows:

Chapter 1 contains the background information and research motivation to understand this thesis.

Chapter 2 contains a Literature Review and identifies the research gaps we address.

Chapter 3 contains the methodology and system architecture used to complete this project.

Chapter 4 contains the results and evaluates the system.

Chapter 5 contains the conclusion of the thesis and highlights future research directions.

Chapter 2

Literature Review

Recent experiments have tested the feasibility of running LLMs on mobile devices such as smartphones, finding that while running LLMs locally is possible from a technical standpoint, latency and high memory use remain challenging issues during actual implementation. The findings are consistent with the challenges experienced in this study involving edge devices, wherein TTFT and high memory usage have a large impact on performance[17].

2.1 Conversational Voice Assistants and their Limitations

Some of the earlier examples of voice assistants include Siri, Alexa, and Google Assistant, which adopt cloud-based models for their working. In these approaches, modeling and speech processing occur on cloud servers, which are not in proximity to the user. Although such methods prove to be very accurate, there are some concerns with network latency and security threats [14]. Some of the attack types include backdoor, hidden-command, adversarial, masquerade, and Dolphin attacks.

2.2 Stream-Based AI-Pipelines for on device systems

Current studies on AI on devices have revealed that there is a need for modular and stream-based pipeline architecture for the deployment of intelligent systems fast and efficiently. The pipeline framework allows the various services of the artificial intelligence to run in parallel as modules[7].

Studies on on-device AI pipelines demonstrate that stream-based architectures enhance performance efficiency, reduce developmental complexity, and enable real-time processing on heterogeneous hardware devices with limited computational capacity[7]. This approach is particularly suitable for conversational systems that require continuous audio processing, incremental transcription, token streaming, and real-time speech synthesis within a unified framework.

2.3 Model Compression Methods

Model compression techniques aim to reduce the size and decrease the compute needed for neural models while preserving their performance. The solution to this problem has three common methods: knowledge distillation, pruning and quantization[15].

In knowledge distillation knowledge, or effectively, weights from large and complex neural nets (also called the 'teacher') are transferred to the a smaller network(called the 'student). The 'student' tries to emulate the output of the larger 'teacher', in effect attempting to perform the same function as the teacher with lesser number of weights[15].

In pruning, connections which are deemed not necessary for the effective inference are removed. This also functions as a type of regularization of the model, increasing generalizability. Extreme use of this methods leads to drastic performance degradation[15].

Quantization is a key optimization strategy for deploying deep learning models on embedded hardware. By representing model weights using lower precision (e.g., 4-bit or 8-bit), quantization significantly reduces memory bandwidth usage and inference latency while preserving acceptable accuracy levels[12][25][1].

Studies on Edge-AI show that the process of compressing and quantizing models is vital when carrying out deep learning workloads on edge devices that have limited processing power, even though it might affect the accuracy of the model slightly. The balancing of accuracy and latency in conversational pipelines is crucial[3].

2.4 Research Gap

According to recent surveys on Edge-AI, executing deep learning tasks at the edge introduces challenges including limited memory, energy constraints, and increased task completion time due to resource scarcity. While edge deployment reduces latency and enhances privacy, the execution of resource-intensive models such as LLMs on thin edge devices remains a major open challenge[3]. Moreover, experiements of LLM deployment on constrained devices highlight high latency and memory bottlenecks, indicating that achieving real-time conversational performance on embedded platforms remains a challenging open problem[17]. Also, despite progress in ASR and LLM deployment, there is limited research on fully streaming, low-latency conversational pipelines operating entirely on edge hardware. Most existing works either rely on cloud LLMs or do not evaluate real-time latency metrics in embedded settings. Additionally, the integration of quantized LLMs in real-time voice systems remains underexplored. Therefore, there is a clear research gap in designing and evaluating a fully offline, low-latency conversational AI pipeline that integrates streaming speech recognition, quantized local LLM inference, and real-time speech synthesis on constrained edge hardware(such as the Raspberry Pi-class or Jetson Nano). This work aims to address this gap by proposing a

modular streaming conversational architecture optimized for real-time edge deployment.

Chapter 3

Methodology and System Architecture

3.1 System Overview

Considering the system needs to be deployed in a contrained embedded device, its architecture has been designed taking into account numerous modern optimizations available. As shown in architecture diagram, the overall system design has some similarities with previous related work, such as the stream-based modular pipeline in the on-device framework such as NNStreamer[7], which demonstrate that stream pipelines improve performance efficiency and modular components allow for rapid prototyping for experimentation.

The user interacts with the system through a microphone input controlled by audio component, which runs in a loop to detect a wakeword. Upon detecting the wakeword, the speech to text transcription system is triggered to record audio upto a period if time till silence is detected, using a neural voice activity detection model[24]. The audio is not held until a turn finishes, but instead chunks based on time heuristic is streamed for transcription to increase responsiveness. The text transcription is aggregated in a queue. Upon detecting the end of a speaker turn, the is created from the transcript and posted to the language model server. The STT model this project is the Whisper[19] model developed by OpenAI. The advantages of this particular ASR model is the multilingual capabilities of this model. By virtue of of the backbone model, it can be extended to support Indian languages such as Hindi, Bengali, etc. using supervised finetuning using the Indic voices dataset[20]. Further the variant selected is the Whisper Large Turbo V3, selected for its robustness, accuracy and moderate size.

For the TTS model, the project selected the Piper neural TTS model from rhasspy[8]. This neural TTS solution is open-source, lightweight, energy efficient and most importantly customizable. This project selects a finetuned version to emulate the Indian-English accent.

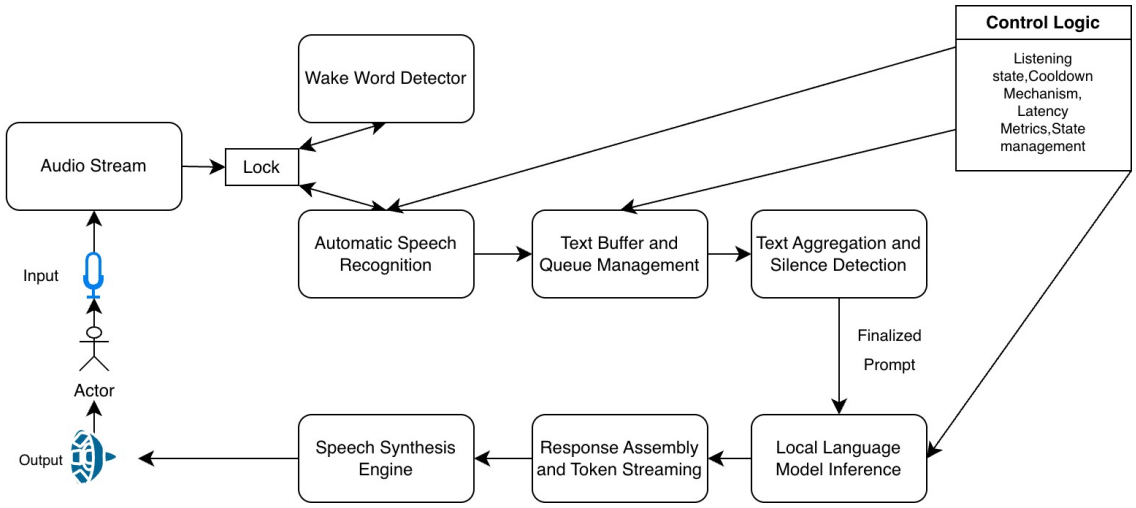

Figure 3.1: Architecture Diagram

3.2 Hardware Setup

The intention of this project is to validate the feasibility of a neural language model based voice assistant system in a device with constrained resources. In this project we consider two edge devices, namely the Raspberry Pi 4B and the high-end Nvidia Jetson Orin Nano. We deploy an extremely optimised version of the assistant on the Raspberry Pi 4B, and see if we can do better with Jetson device since it is equipped with matrix operation accelarators for AI applications.

As shown in the table, the Pi has 4GB of RAM, which poses a challenge to deploy the system. The Jetson device on the other hand with 8GB of memory and Cuda cores should be able to comfortably handle the system. Both hardware have binned low power consumption. A Logitech device with microphone and speaker is selected as the interface.

Table 3.1: Raspberry Pi, Jetson Nano hardware specifications.

Component	**Specification**	
Name	Raspberry Pi 4B	Nvidia Orin Jetson Nano
Memory	4GB LPDDR4	8GB LPDDR5
CPU	4-core Arm®-A76 64-bit CPU	6-core Arm®-A78AE 64-bit CPU
GPU	VideoCore VII GPU	Ampere architecture with 1024 CUDA cores
Power	10W to 27W	7W to 25W

Algorithm 1: Event-Driven Streaming Conversational Pipeline

Input : Continuous audio stream
Output: Spoken conversational response

Initialize audio interface, buffer queue, and control state;
Initialize latency monitoring variables;
Start audio processing thread;
Start aggregation and decision thread;
while *system is active* **do**
  if *wakeword detected* **then**
    Capture incoming audio frames;
    Convert audio into incremental text segments;
    Enqueue text segments into shared buffer;
    if *interaction module is idle* **then**
      Dequeue available text segments;
      Append segments to temporary buffer;
      Update last activity timestamp;
    if *silence duration exceeds predefined threshold* **then**
      Finalize buffered text as user prompt;
      if *prompt length satisfies minimum criteria* ***and*** *prompt is novel* **then**
        Record speech termination timestamp;
        Disable listening state to prevent feedback;
        Forward prompt to inference module;
        Stream generated response tokens;
        Record time-to-first-token and throughput metrics;
        Aggregate tokens into final response text;
        Perform on-device speech synthesis;
        Apply cooldown and re-enable listening;
        Reset buffer and internal interaction state;
      else
        Discard buffer and continue listening;

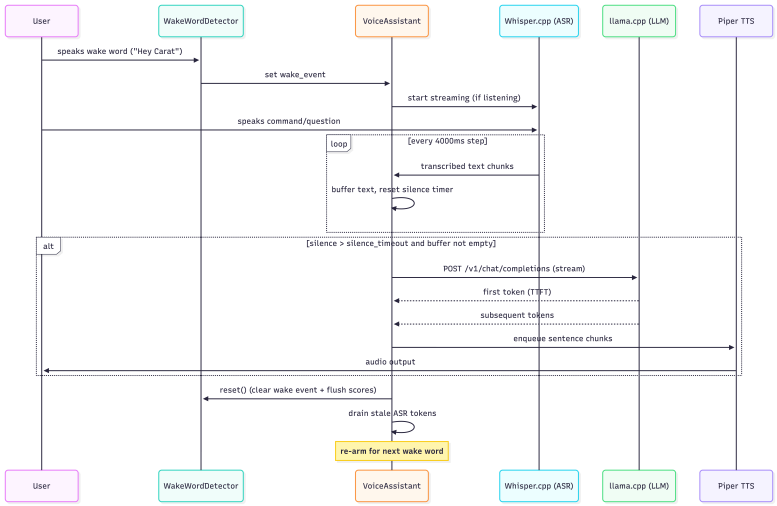

Figure 3.2: Sequence Diagram of the Event-Driven Conversational Pipeline

3.3 Software Stack

For certain components of the assistant, open-source software code sped up the prototyping phase. We have utilised whisper.cpp for Streaming ASR[19][5]. This features a higly optimsed inference-only implementation of the Whisper model with capabilities to run sufficiently well on CPU-only or even utilize accelarators. Similarly, the system employs a CPU optimized LLM inference backend based on llama.cpp, with optional GPU inference, which supports GGUF(GPT-Generated Unified Format) formatted quantized models and efficient token streaming suitable for real time edge deployment[4]. For Text to Speech, Piper by rhasspy[8], for central coordination we use Python (threading and pipeline control) and the Requests API (LLM streaming interface).

3.3.1 Wake Word Detection

This approach uses a customized wake word detection system to trigger the conversation. The openWakeWord framework [21] was adopted in designing the system, which uses a light-weighted neural network model that works well on hardware with limited processing capacity. A custom model was built especially for the wake word “Hey Carat”. The training data was generated by the Piper TTS synthesis engine, adding variability in prosody, ambient noise, and speaker identity. The final custom model attained 92% accuracy during validation without false positives.

3.3.2 Streaming ASR Integration

As described, the audio stream from the microphone is gated using a wake-word model[21]. The whisper.cpp[5] binary executable is executed as a subprocess and its output is parsed line by line. A queue-based multithreaded data structure is used to buffer transcribed text segments. A silence based Voice Activity Detection model is utilized to detect a speaker turn[24] and create the final prompt.

3.3.3 Local LLM Inference using llama.cpp

The system uses a locally hosted LLM HTTP server with the Qwen2.5-0.5B-Instruct[18] model quantized to 4-bit precision using smart blockwise quantization on the Raspberry Pi. On the Jetson Nano the Gemma3-1B-instruct[22] model is selected with the same quantization technique. Streaming token generation is used to reduce latency and improve perceived responsiveness.

3.3.4 Text-to-Speech Integration

For on-device speech synthesis, Piper[8] is integrated as a lightweight TTS solution. The audio is generated after chunked responses reach a mimimum length during LLM inference. This is in line with the streaming design decision used multiple times in this project.

3.4 Language model selection

Due to resource constraints, it is important to justify the rationale for choosing a particular LM. There needs to be a compromise between parameter counts(higher usually implies better performance), net RAM usage,etc. Additionaly, we also list mutimodal variants of the LMs with vision capabilities, but the additional overheads posed due to having multiple encoders deters the use of such models for constrained environments. As a result, the actual contenders are limited to text-generation-only models. Based off of these criteria, some candidate models were:

Table 3.2: Memory and Performance Suitability of Candidate Models for Edge Deployment

Model	Parameters	Approx. 4-bit version RAM Usage	Performance / Perplexity Proxy	Edge Feasibility
Qwen3-VL-2B-Instruct	2B	∼1.2–1.8 GB	Moderate	Moderate
PaliGemma 3B	3B	∼2–3 GB	High	Feasible
MiniCPM-V 2B	2B	∼1.3–1.8 GB	Moderate	Moderate
Gemma3 1B	1B	∼0.7–1.1 GB	Low-Moderate	Excellent
Qwen2.5-0.5B (Instruct)	0.5B	∼0.4–0.7 GB	Low	Excellent
7B-class VLMs	7B	∼5–8 GB	Very High	Low

This project selected the Qwen2.5-0.5B model for the assistant on Raspberry Pi and the Gemma3-1B on the Jetson Nano. Models with parameters in the 0.5-1B parameter ranges are able to generate meaningful sentences given the context. The Qwen2.5 family of models were released by Alibaba and perform much better in reasoning tasks and text generation than their predecessors. The 0.5B param model, when quantized, performs fast inference on the Pi, making it particularly suitable for our task. The Gemma3 class of models developed by Google, were claimed to be trained for deployment in end-consumer devices. The difference over the last generation is reduced model sizes with similar capabilities, which makes it suitable for deployment on the Jetson Nano with its accelarator GPU cores. We do not consider the other models due to either slower inference, or higher perplexity.

3.5 Quantization Method

From a performance perspective, the use of lower-bit quantization directly contributes to improved interaction metrics, including reduced Time to First Token, and higher tokens per second. Since the conversational pipeline involves multiple non trivial modules, like the streaming ASR, LLM inference engine, and TTS synthesis, minimizing model memory

access and computational overhead is critical for maintaining responsiveness. Consistent with expectations, it can be seen from our experiment during deployment that there is a direct correlation between increased inference latency and memory consumption due to high-precision models (FP16 bits/8 bits), while the 4-bit quantization model has consistently performed throughputs within the physical capabilities of our edge devices. It can be said that low-bit quantization is more desirable than high-bit full-precision models since they do not provide any significant improvements in perplexity or benchmark accuracy[13].

Based on that, the quantization technique applied here is the post-training weight quantization via the llama.cpp [4] inference engine. In other words, for both of them, GGUF model format along with the Q4_K_M quantization approach is applied. The quantization approach is not the typical uniform one since, in addition to per-block scaling, there are optimizations with respect to the weight layout. Such an approach not only enables achieving the same results but also leads to significant memory savings, which makes it possible to perform the model inference entirely on CPU.

3.5.1 Overview of Q4_K_M Quantization

The Q4_K_M format is a highly optimized, mixed-precision quantization strategy developed within the llama.cpp framework (part of the "k-quants" family). It is widely regarded as the most adequate quantization technique for running large language models locally, offering a balance between model compression (VRAM/RAM footprint) and generation quality (perplexity).

The nomenclature of Q4_K_M can be broken down into three components:

- **q4**: The baseline quantization targets 4 bits per weight.
- **K**: Denotes the "k-quants" block structure, which uses hierarchical super-blocks to maintain numerical precision.
- **M**: Stands for "Medium." It indicates a mixed-precision profile where the most the tensors are quantized to 4-bit (Q4_K), but highly sensitive tensors are retained at 6-bit (Q6_K) precision.

3.5.2 The Q4_K Block Architecture

In contrast to standard block quantization (such as Q4_0) which simply groups 32 weights with a single 32-bit floating-point scale, the `K`-quantization utilizes a hierarchical design to minimize precision loss.

In Q4_K, weights are grouped into a super-block of $N = 256$ weights. This super-block is further subdivided into 8 sub-blocks of 32 weights each. The structure of a Q4_K super-block is defined as: The 256 weights are stored as 4-bit integers.Instead of storing full

FP16 or FP32 scales for each sub-block, the scale and minimum values for the 8 sub-blocks are themselves quantized to 6-bit integers. A single higher-precision scale (usually FP16) is stored for the entire 256-weight super-block to scale the 6-bit sub-block scales back to their true floating-point ranges.

Mathematically, the dequantization of a given weight w_i within a sub-block j of a super-block can be approximated by:

$$w_i \approx \Delta_{\text{super}} \times (s_j \times q_i + m_j) \tag{3.1}$$

where:

- q_i is the 4-bit quantized weight.
- s_j and m_j are the 6-bit scale and minimum value for the j-th sub-block.
- Δ_{super} is the FP16 super-scale factor for the entire 256-weight block.

Because the overhead of the scales is amortized over 256 weights while maintaining tight 32-weight local scaling, the effective bit-rate of Q4_K is approximately 4.5 bits per weight.

3.5.3 The Medium(M) Mixed-Precision Profile

Not all layers in a transformer architecture are equally sensitive to quantization noise. The "M" profile selectively applies higher precision to the tensors that have the greatest impact on perplexity.

In the `q4_K_M` profile:

- **Attention and Feed-Forward layers:** Half of the `attention.wv` (value projection) and `feed_forward.w2` (down projection) tensors are quantized using the denser `Q6_K` format (6-bit quantization).
- **Remaining Tensors:** All other standard weight matrices are quantized using the baseline Q4_K format.

3.5.4 Performance Trade-offs

By isolating the most sensitive computational pathways and allocating them 6 bits, Q4_K_M manages to achieve perplexity scores that are practically indistinguishable from uncompressed FP16 models. The resulting model is approximately a quarter of the size of its FP16 counterpart and allows standard consumer hardware to load and execute large neural networks efficiently, while avoiding the steep intelligence degradation that is typically associated with flat 4-bit quantization.

Chapter 4

Results and Evaluation

4.1 Evaluation of LM responses

As a simple measure of the change in model responses due to quantization, we measure the perplexity score on the WikiText-V2 [16] dataset on both the models for different quantization levels and the relative changes that result. Perplexity (PPL) evaluates how well a probability model predicts a sample, and for a sequence of tokens $X = (x_1, x_2, \ldots, x_N)$, it is defined as the exponentiated average negative log-likelihood:

$$PPL(X) = \exp\left(-\frac{1}{N}\sum_{i=1}^{N}\log P(x_i|x_{<i})\right) \tag{4.1}$$

where N is the total number of tokens in the sequence, x_i is the i-th token, $x_{<i}$ represents all preceding context tokens, and $P(x_i|x_{<i})$ is the probability the language model assigns to the actual token x_i given that context. It is important to note that raw perplexity scores are not comparable across different models because of the difference in tokenizers used. They are only useful for comparing the variants of a model with itself. The raw scores as well as the comparison to the full precision score are presented for the selected model in Table 4.1.

The analysis conducted using the Qwen 2.5 0.5B model through a 512-token context window gives insights into the quantization process pertinent to on-device execution. For the unquantized model that operates on FP16 precision, the resulting PPL is recorded at 13.90. By quantizing the weights and parameters to 8-bit format (Q8_0), the PPL rises by 13.36% to 15.76, signifying the worst-case drop in performance in the entire quantization sequence. Further compression into 4-bit (Q4_K_M) format brings about an aggregate relative change in the order of 17.09% (PPL 16.28), but the incremental reduction in quality incurred when compressing to 4-bit format after 8-bit is almost negligible (+0.52 PPL). For limited-resource edge devices like the Raspberry Pi, the Q4_K_M configuration presents the best compromise between efficient memory usage and model performance.

We use the Q4_K_M quantized method directly on the Gemma3-1B model deployed in the Jetson Nano. Here we do not bother tuning thread counts and directly allocate 2 threads to the inference engine as the Nano uses CUDA cores for GPU inference unlike the Raspberry

Table 4.1: Perplexity Degradation for Qwen2.5-0.5B Instruct on WikiText-V2

Model / Precision	Model Size	Perplexity (PPL)	Absolute Increase	% Increase (vs FP16)
FP16 (Baseline)	1.27 GB	13.90 ± 0.10	—	—
Q8_0 (8-bit)	676 MB	15.76 ± 0.12	+ 1.86	+ 13.36%
Q4_0 (4-bit)	429 MB	17.45 ± 0.13	+ 3.55	+ 25.53%
Q4_K_M (Selected)	**491 MB**	$\mathbf{16.28 \pm 0.12}$	**+ 2.38**	**+ 17.09%**

Note: Lower perplexity scores indicate better model performance. The Q4_K_M variant is selected as it provides the critical memory and compute reductions necessary for the target hardware while incurring only a marginal additional quality loss compared to the Q8_0 format.

Pi. Two threads are sufficient for decent performance based on our tests, and we get token generation speed upwards of 5 TPS.

Table 4.2: llama.cpp Performance for Qwen2.5-0.5B Instruct on Raspberry Pi 4B(CPU-only)

Quantization	Size (MiB)	Threads	PP50 (t/s)	TG50 (t/s)
BF16 (Unquantized)	942.43	2	9.09 ± 0.01	5.27 ± 0.01
		3	9.08 ± 0.07	5.36 ± 0.08
		4	9.05 ± 0.06	5.36 ± 0.05
Q8_0 (8-bit)	638.74	2	10.13 ± 0.03	5.48 ± 0.04
		3	10.11 ± 0.06	5.47 ± 0.05
		4	10.05 ± 0.06	5.44 ± 0.03
Q4_K_M (4-bit)	462.96	**2**	$\mathbf{9.00 \pm 0.04}$	$\mathbf{7.00 \pm 0.01}$
		3	8.97 ± 0.03	7.00 ± 0.02
		4	8.95 ± 0.04	6.94 ± 0.01

Note: PP50 represents Prompt Processing speeds for 50 tokens and TG50 represents Token Generation speeds(50 tokens). Adding threads beyond 2 generally results in flat or slightly diminished performance on this hardware configuration.

4.2 Evaluation Protocol

The system is evaluated based on conversational performance and latency measurements during live voice interaction sessions. A typical 50 token sample prompt is delivered and the metrics are measured. The key metrics are the Time to First Token(TTFT)(a proxy for responsiveness), Tokens per Second(TPS) and a subjective evaluation of the answer content based on the context.

4.2.1 LLM Inference Performance on Edge Devices

The performance of the local LLM was evaluated on both edge platforms using a sample prompt of approximately 25 tokens. The Raspberry Pi 4B used the Qwen2.5-0.5B-Instruct model, while the Jetson Orin Nano used the Gemma3-1B-Instruct model, both quantized to Q4_K_M.

Table 4.3: Inference Performance for Gemma 3 1B on NVIDIA Jetson Orin Nano(Average of 5 Runs)

Model	Quantization	Prompt Processing (t/s)	Token Generation (t/s)
Gemma 3 1B	**Q4_K_M (4-bit)**	**7.77**	**5.27**

Note: Performance metrics recorded locally on the NVIDIA Jetson Orin Nano. Prompt Processing (PP) refers to the ingestion speed, while Token Generation (TG) refers to the continuous output speed.

Table 4.4: LLM Performance Comparison on Edge Devices (25-token prompt)

Metric	Raspberry Pi 4B	Jetson Nano
Model Name	Qwen2.5-0.5B-Q4_K_M	Gemma3-1B-Q4_K_M
Prompt Processing Speed	9.0 ± 0.45	7.77 ± 0.28
Token Generation Speed	7.0 ± 0.35	5.27 ± 0.22
Time to First Token (TTFT)	5.5 ± 1.3	4.11 ± 0.45
End-to-End Response Time*	18 ± 2.8	14 ± 1.4

*End-to-end time includes streaming ASR, LLM inference, response assembly, and TTS synthesis. Values are reported as Mean $\pm$ Standard Deviation over multiple runs.

Due to limitations such as the absence of hardware acceleration, the Raspberry Pi 4B has more latency than the Jetson Orin Nano, which has a 4-core CPU. However, the lightweight 0.5B Raspberry Pi is capable of producing practical speech conversation, while the more responsive Jetson can produce more natural conversation.

4.2.2 Memory Profiling and Resource Utilization

As the conversational model is executed on the edge device, the efficient use of memory plays a significant role in ensuring stability, responsiveness, and practicality of the system. Memory usage testing was performed using the system in action while streaming speech recognition, local large language model inference, and text-to-speech synthesis were executing simultaneously.

The results from practical experiments on a Raspberry Pi 4B indicate that the maximum memory used by the entire system in steady-state operations stayed around 1.8–2.2 GB. Memory usage by the different modules is depicted in Table 4.5.

Table 4.5: Runtime Memory Utilization of System Components on Raspberry Pi 4B (4 GB RAM)

Component	Approximate Memory Usage
Local LLM (Qwen2.5-0.5B-Instruct, Q4_K_M)	∼398–650 MB
Streaming ASR (whisper.cpp - Medium English)	∼700–950 MB
TTS (Piper)	∼250–400 MB
System Overhead (Python threads, buffers, queues, OS)	∼200–350 MB
Total Peak Memory Usage	∼1.8–2.2 GB

Based on the memory analysis, it is evident that the Streaming ASR component (whisper.cpp Medium model) currently makes up a major part of the memory allocation, followed by the LLM inference model. Due to the very small footprint of the Qwen2.5-0.5B model (∼398 MB upon quantization), there is enough memory left for its execution under the memory constraint of the 4 GB RAM of the Raspberry Pi 4B device.

Despite being less efficient in terms of RAM usage, the Piper neural TTS offers better speech quality than lightweight synthesizers. It should be noted that despite the presence of several processes working simultaneously and storing audio data in memory, the system was still able to work within its memory resources efficiently.

4.3 System performance on Edge Hardware

The system successfully executes the entire conversational pipeline without any dependence on external clouds. The quantization of the 4-bit LLM helps perform consistent inference under constraints, producing responses that are appropriate for use in voice assistants.

This shows that full-fledged conversational AI can be achieved even using constraint-based hardware. Token stream generation enhances the perceived responsiveness of the model, whereas silence segmentation enables smooth conversation flow.

4.4 Limitations

Even though the conversational pipeline has been implemented successfully, there are several constraints that have to be resolved. For instance, there are limitations in terms of the model's size because of the limited computing resources and memory available on the edge device. The Qwen2.5-0.5B-Instruct model does not exceed the memory available on the Raspberry Pi 4B. However, the small number of parameters is one of the main drawbacks of the model because of the inability to perform in-depth reasoning and contextual intelligence when compared with 7B-class models. Also, the absence of any hardware acceleration on the Raspberry Pi 4B leads to a need for CPU-based inference, which limits the performance in terms of tokens per second.

There is also an issue related to the quality of the audio that is synthesized. Though the Piper neural TTS engine provides an outstanding compromise between fast processing and quality when deployed on edge, the voices created by it usually have relatively poor prosody as opposed to the high compute counterparts that work in cloud-based neural systems. The application of lightweight vocoders and model compression that allows maintaining real-time processing on edge devices such as Jetson Nano and Raspberry Pi can sometimes lead to robotic intonation and phonetic glitches.

Furthermore, the strength of the speech recognition and dialogue flow logic is constrained by certain environmental conditions. The present voice activity detection

algorithm based on silence utilizes time heuristics, which may become a source of malfunctioning when the noise levels are too high or when there is a pause between phrases. In addition, even though the specialized wake word model has been shown to have good accuracy rates, it requires an adequately working microphone and close proximity to the user since otherwise, false negatives will occur from time to time, interrupting smooth conversations. Finally, being entirely offline and not connected to the web, the model cannot obtain any up-to-date information except that contained within itself.

Chapter 5

Conclusion and Future Work

5.1 Conclusion

This dissertation successfully proves the feasibility of designing an end-to-end conversational AI system completely based on edge devices. Through the development of a multi-threaded pipeline containing ASR with streaming processing, quantized LLM inference, and TTS with neural nets, all the major drawbacks of cloud-based systems such as latency, privacy, and dependence on internet connection are effectively solved.

The experiments conducted on Raspberry Pi 4B and NVIDIA Jetson Orin Nano indicate the possibility of performing conversational AI tasks in real-time without any cloud assistance. More precisely, using the approach of $Q4\_K\_M$ quantization allowed maintaining the stability of the system despite strict memory limitations while retaining enough speed and accuracy to provide natural conversation. Overall, this research should be considered a basis for future projects in private sensing and human-robot interaction in edge environment.

5.2 Future Work

An important direction for further investigation is concerned with the thorough SFT of the models using curated conversation data sets. The goal would be to use the instruction-following dialogue corpus to increase response relevance and coherence by focusing on the intelligent characteristics of the models instead of the latency-focused metrics currently used. Another future improvement would be an expansion of text-only models to multimodal models that can perform visual processing and thus have a wider range of environmental interactions. Some future improvements will also include addressing issues with audio and dialogue logic. One important change is the transition from current simple VAD heuristics to a more accurate neural Voice Activity Detection to increase accuracy in noisy conditions. The other notable step forward is the implementation of lightweight vocoders to improve the naturalness of the synthesized speech to solve the problem with robotic intonation when running on edge devices. Finally, implementing the dialogue

pipeline with ROS would be huge milestone in improving human-robot interaction.

References

[1] Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. Llm.int8(): 8-bit matrix multiplication for transformers at scale, 2022.

[2] Reece H. Dunn et al. espeak ng: A multi-lingual software speech synthesizer. https://github.com/espeak-ng/espeak-ng, 2024.

[3] Himanshu Gauttam, Garima Nain, K.K. Pattanaik, and Paulo Mendes. Edge-ai: A systematic review on architectures, applications, and challenges. *Journal of Network and Computer Applications*, 245:104375, 2026.

[4] Georgi Gerganov. llama.cpp: Inference of large language models in c/c++. https://github.com/ggerganov/llama.cpp, 2023. Accessed: March 2026.

[5] Georgi Gerganov. whisper.cpp: Port of openai's whisper model in c/c++. https://github.com/ggerganov/whisper.cpp, 2023. Accessed: 2026-03.

[6] Swathi Gowroju, Sandeep Kumar, and Shilpa Choudhary. Natural language processing-driven voice recognition system for enhancing desktop assistant interactions. In *2024 7th International Conference on Contemporary Computing and Informatics (IC3I)*, volume 7, pages 1136–1141, 2024.

[7] MyungJoo Ham, Sangjung Woo, Jaeyun Jung, Wook Song, Gichan Jang, Yongjoo Ahn, and Hyoung Joo Ahn. Toward among-device ai from on-device ai with stream pipelines, 2022.

[8] Michael Hansen. Piper: A fast, local neural text-to-speech system. https://github.com/rhasspy/piper, 2023.

[9] Fred Hohman, Mary Beth Kery, Donghao Ren, and Dominik Moritz. Model compression in practice: Lessons learned from practitioners creating on-device machine learning experiences. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*, CHI '24, New York, NY, USA, 2024. Association for Computing Machinery.

[10] Matthew B. Hoy. Alexa, siri, cortana, and more: An introduction to voice assistants. *Medical Reference Services Quarterly*, 37(1):81–88, 2018. PMID: 29327988.

[11] Arun Jagatheesan, Jong Hoon Ahnn, Thomas Phan, Abhishek Singh, and Juhan lee. Hierarchical automatic speech recognition powered by data infrastructure. 01 2014.

[12] Renren Jin, Jiangcun Du, Wuwei Huang, Wei Liu, Jian Luan, Bin Wang, and Deyi Xiong. A comprehensive evaluation of quantization strategies for large language models. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar, editors, *Findings of the Association for Computational Linguistics: ACL 2024*, pages 12186–12215, Bangkok, Thailand, August 2024. Association for Computational Linguistics.

[13] Renren Jin, Jiangcun Du, Wuwei Huang, Wei Liu, Jian Luan, Bin Wang, and Deyi Xiong. A comprehensive evaluation of quantization strategies for large language models, 2024.

[14] Jingjin Li, Chao Chen, Mostafa Rahimi Azghadi, Hossein Ghodosi, Lei Pan, and Jun Zhang. Security and privacy problems in voice assistant applications: A survey. *Computers Security*, 134:103448, 2023.

[15] Xiaomei Li, Mei Tang, and Shan Xiao. Research on model compression and efficient inference algorithm for deep neural networks. In *2025 International Conference on Electrical Drives, Power Electronics Engineering (EDPEE)*, pages 1030–1035, 2025.

[16] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models, 2016.

[17] Mateus Monteiro Santos, Aristoteles Barros, Luiz Rodrigues, Diego Dermeval, Tiago Primo, Ig Ibert, and Seiji Isotani. Near feasibility, distant practicality: Empirical analysis of deploying and using llms on resource-constrained smartphones. In *Proceedings of the 13th International Conference on Information & Communication Technologies and Development*, ICTD '24, page 224–235, New York, NY, USA, 2025. Association for Computing Machinery.

[18] Qwen, :, An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tianyi Tang, Tingyu Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yu Wan, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zihan Qiu. Qwen2.5 technical report, 2025.

[19] Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever. Robust speech recognition via large-scale weak supervision, 2022.

[20] Ashwin Sankar, Srija Anand, Praveen Srinivasa Varadhan, Sherry Thomas, Mehak Singal, Shridhar Kumar, Deovrat Mehendale, Aditi Krishana, Giri Raju, and Mitesh M. Khapra. Indicvoices-r: Unlocking a massive multilingual multi-speaker speech corpus for scaling indian TTS. In Amir Globersons, Lester Mackey, Danielle Belgrave, Angela Fan, Ulrich Paquet, Jakub M. Tomczak, and Cheng Zhang, editors, *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*, 2024.

[21] David Scripka. openwakeword: Open-source wake word detection. `https://github.com/dscripka/openWakeWord`, 2023.

[22] Gemma Team. Gemma 3 technical report, 2025.

[23] Qwen Team. Qwen3 technical report, 2025.

[24] Silero Team. Silero models: pre-trained text-to-speech models made embarrassingly simple. `https://github.com/snakers4/silero-models`, 2025.

[25] Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. Smoothquant: Accurate and efficient post-training quantization for large language models, 2024.

Mainak Ghosh

DissertationGuidelinesV4_5-6-30

RP_Mid_MTech_26

Document Details

Submission ID
trn:oid:::3618:138340518

Submission Date
May 10, 2026, 10:43 AM GMT+5:30

Download Date
May 10, 2026, 11:03 AM GMT+5:30

File Name
DissertationGuidelinesV4_5-6-30.pdf

File Size
930.9 KB

25 Pages

6,495 Words

36,656 Characters

*% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?
The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?
Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.

Mainak Ghosh

DissertationGuidelinesV4_5-6-30

RP_Mid_MTech_26

Document Details

Submission ID
trn:oid:::3618:138340518

Submission Date
May 10, 2026, 10:43 AM GMT+5:30

Download Date
May 10, 2026, 10:58 AM GMT+5:30

File Name
DissertationGuidelinesV4_5-6-30.pdf

File Size
930.9 KB

25 Pages

6,495 Words

36,656 Characters

9% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography

Match Groups

55 Not Cited or Quoted 9%
Matches with neither in-text citation nor quotation marks

0 Missing Quotations 0%
Matches that are still very similar to source material

0 Missing Citation 0%
Matches that have quotation marks, but no in-text citation

0 Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

4% Internet sources
3% Publications
7% Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.